Demographic Feature Analysis for Diabetes Risk

AUTHOR
Bereket Tarekegn

Project Overview

This section of the ADS 503 final project focuses on demographic and socioeconomic predictors of diabetes risk using the CDC BRFSS 2015 Diabetes Health Indicators dataset.

The original outcome variable, `Diabetes_012`, has three classes: no diabetes, prediabetes, and diabetes. For this analysis, I converted the outcome into a binary screening target:

- `NoRisk` = no diabetes
- `AtRisk` = prediabetes or diabetes

This approach fits the project goal because the purpose is early screening. The model is not intended to diagnose diabetes. Instead, it is designed to identify people who may need follow-up screening or preventive care.

Load Packages

```
# Install packages first if needed:
# install.packages(c("tidyverse", "caret", "knitr", "rpart", "randomForest", "pROC"))

library(tidyverse)
library(caret)
library(knitr)
library(rpart)
library(randomForest)
library(pROC)

set.seed(503)
```

Load Dataset

```
csv_path <- "C:/Project/diabetes_012_health_indicators_BRFSS2015.csv"

if (!file.exists(csv_path)) {
  csv_path <- "diabetes_012_health_indicators_BRFSS2015.csv"
}

if (!file.exists(csv_path)) {
  stop("Dataset not found. Please put diabetes_012_health_indicators_BRFSS2015.csv in C:/Projec
}

diabetes <- read.csv(csv_path)

dim(diabetes)
```

```
[1] 253680     22
```

```
names(diabetes)
```

```
 [1] "Diabetes_012"         "HighBP"               "HighChol"
 [4] "CholCheck"            "BMI"                  "Smoker"
 [7] "Stroke"               "HeartDiseaseorAttack" "PhysActivity"
[10] "Fruits"               "Veggies"              "HvyAlcoholConsump"
[13] "AnyHealthcare"        "NoDocbcCost"          "GenHlth"
[16] "MentHlth"             "PhysHlth"             "DiffWalk"
[19] "Sex"                  "Age"                  "Education"
[22] "Income"
```

The dataset contains 253,680 observations and 22 variables from the CDC BRFSS 2015 survey.

Create Binary Diabetes Risk Target

```r
demo_data <- diabetes %>%
  mutate(
    Diabetes_Risk = ifelse(Diabetes_012 == 0, "NoRisk", "AtRisk"),
    Diabetes_Risk = factor(Diabetes_Risk, levels = c("AtRisk", "NoRisk"))
  ) %>%
  dplyr::select(
    Diabetes_Risk,
    Age,
    Sex,
    Education,
    Income,
    AnyHealthcare,
    NoDocbcCost
  )

str(demo_data)
```

```
'data.frame':   253680 obs. of  7 variables:
 $ Diabetes_Risk: Factor w/ 2 levels "AtRisk","NoRisk": 2 2 2 2 2 2 2 2 1 2 ...
 $ Age          : num  9 7 9 11 11 10 9 11 9 8 ...
 $ Sex          : num  0 0 0 0 0 1 0 0 0 1 ...
 $ Education    : num  4 6 4 3 5 6 6 4 5 4 ...
 $ Income       : num  3 1 8 6 4 8 7 4 1 3 ...
 $ AnyHealthcare: num  1 0 1 1 1 1 1 1 1 1 ...
 $ NoDocbcCost  : num  0 1 1 0 0 0 0 0 0 0 ...
```

```r
summary(demo_data)
```

```
 Diabetes_Risk        Age              Sex           Education     
 AtRisk: 39977   Min.   : 1.000   Min.   :0.0000   Min.   :1.00  
 NoRisk:213703   1st Qu.: 6.000   1st Qu.:0.0000   1st Qu.:4.00  
                 Median : 8.000   Median :0.0000   Median :5.00  
                 Mean   : 8.032   Mean   :0.4403   Mean   :5.05  
                 3rd Qu.:10.000   3rd Qu.:1.0000   3rd Qu.:6.00  
                 Max.   :13.000   Max.   :1.0000   Max.   :6.00  
     Income      AnyHealthcare     NoDocbcCost      
 Min.   :1.000   Min.   :0.0000   Min.   :0.00000  
```

```
 1st Qu.:5.000   1st Qu.:1.0000   1st Qu.:0.00000
 Median :7.000   Median :1.0000   Median :0.00000
 Mean   :6.054   Mean   :0.9511   Mean   :0.08418
 3rd Qu.:8.000   3rd Qu.:1.0000   3rd Qu.:0.00000
 Max.   :8.000   Max.   :1.0000   Max.   :1.00000
```

The target variable is now binary. Prediabetes and diabetes are combined into the `AtRisk` class.

Recode Predictors

```
demo_data <- demo_data %>%
  mutate(
    Sex = factor(Sex, levels = c(0, 1), labels = c("Female", "Male")),
    Age = factor(Age),
    Education = factor(Education),
    Income = factor(Income),
    AnyHealthcare = factor(AnyHealthcare, levels = c(0, 1), labels = c("No", "Yes")),
    NoDocbcCost = factor(NoDocbcCost, levels = c(0, 1), labels = c("No", "Yes"))
  )

str(demo_data)
```

```
'data.frame':   253680 obs. of  7 variables:
 $ Diabetes_Risk: Factor w/ 2 levels "AtRisk","NoRisk": 2 2 2 2 2 2 2 2 1 2 ...
 $ Age          : Factor w/ 13 levels "1","2","3","4",..: 9 7 9 11 11 10 9 11 9 8 ...
 $ Sex          : Factor w/ 2 levels "Female","Male": 1 1 1 1 1 2 1 1 1 2 ...
 $ Education    : Factor w/ 6 levels "1","2","3","4",..: 4 6 4 3 5 6 6 4 5 4 ...
 $ Income       : Factor w/ 8 levels "1","2","3","4",..: 3 1 8 6 4 8 7 4 1 3 ...
 $ AnyHealthcare: Factor w/ 2 levels "No","Yes": 2 1 2 2 2 2 2 2 2 2 ...
 $ NoDocbcCost  : Factor w/ 2 levels "No","Yes": 1 2 2 1 1 1 1 1 1 1 ...
```

```
summary(demo_data)
```

```
 Diabetes_Risk        Age            Sex         Education      Income
 AtRisk: 39977   9      :33244   Female:141974   1:   174   8      :90385
 NoRisk:213703   10     :32194   Male  :111706   2:  4043   7      :43219
```

```
                8      :30832                  3:  9478   6      :36470
                7      :26314                  4: 62750   5      :25883
                11     :23533                  5: 69910   4      :20135
                6      :19819                  6:107325   3      :15994
                (Other):87744                             (Other):21594
AnyHealthcare NoDocbcCost
No : 12417    No :232326
Yes:241263    Yes: 21354
```

These variables are treated as categorical features because they represent survey response groups.

Missing Value Analysis

```
missing_table <- data.frame(
  Variable = names(demo_data),
  Missing_Count = colSums(is.na(demo_data)),
  Missing_Percent = round(colSums(is.na(demo_data)) / nrow(demo_data) * 100, 2)
)

kable(missing_table, caption = "Missing Values for Demographic Feature Group")
```

Missing Values for Demographic Feature Group

	Variable	Missing_Count	Missing_Percent
Diabetes_Risk	Diabetes_Risk	0	0
Age	Age	0	0
Sex	Sex	0	0
Education	Education	0	0
Income	Income	0	0

	Variable	Missing_Count	Missing_Percent
AnyHealthcare	AnyHealthcare	0	0
NoDocbcCost	NoDocbcCost	0	0

The selected demographic variables do not contain missing values, so no imputation is needed for this feature group.

Class Distribution

```
target_table <- demo_data %>%
  count(Diabetes_Risk) %>%
  mutate(Percent = round(n / sum(n) * 100, 2))

kable(target_table, caption = "Binary Diabetes Risk Class Distribution")
```

Binary Diabetes Risk Class Distribution

Diabetes_Risk	n	Percent
AtRisk	39977	15.76
NoRisk	213703	84.24

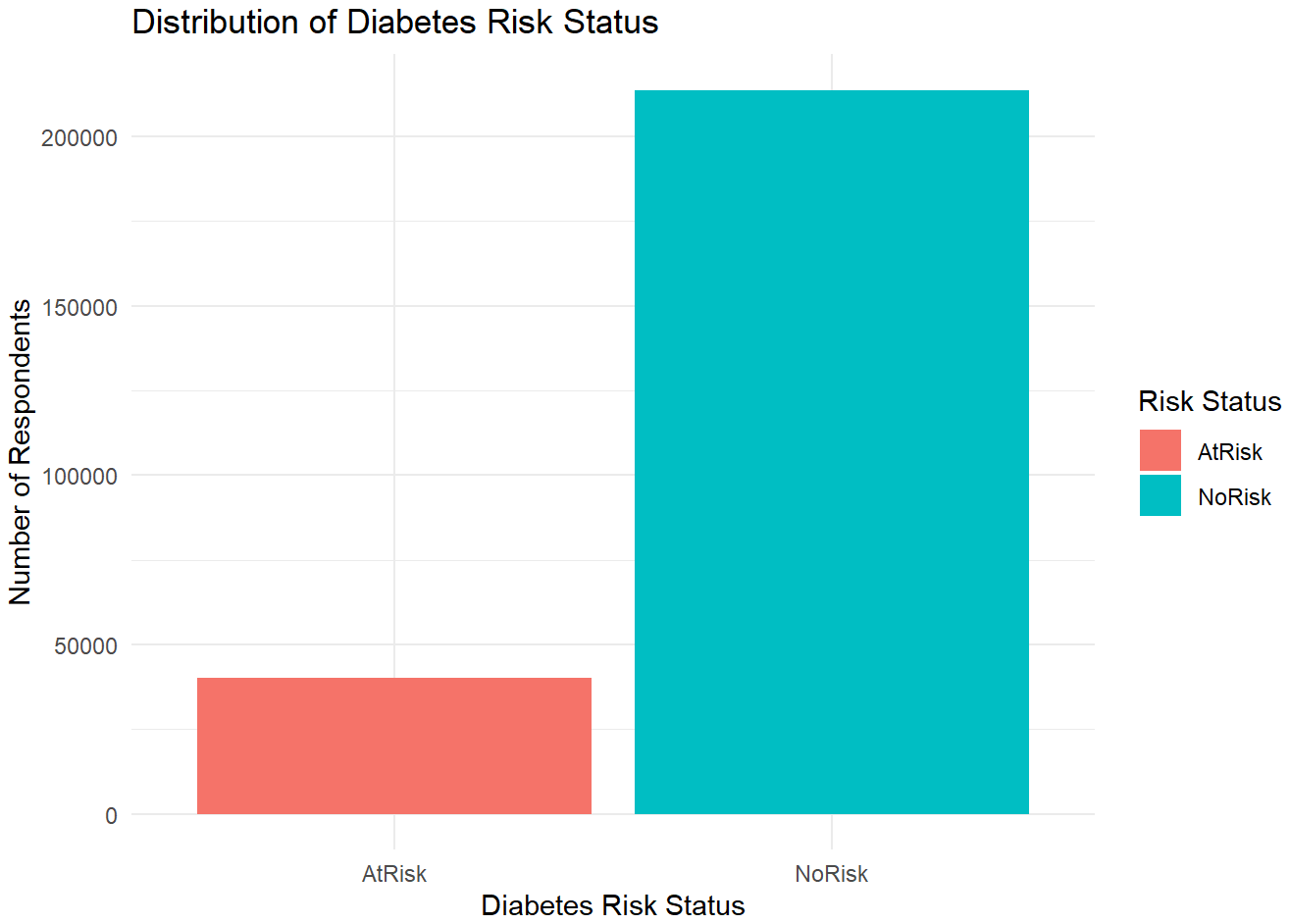
The class distribution is imbalanced. There are more respondents in the `NoRisk` group than in the `AtRisk` group. Because of this imbalance, accuracy alone is not enough to judge model performance.

Exploratory Data Analysis

Diabetes Risk Distribution

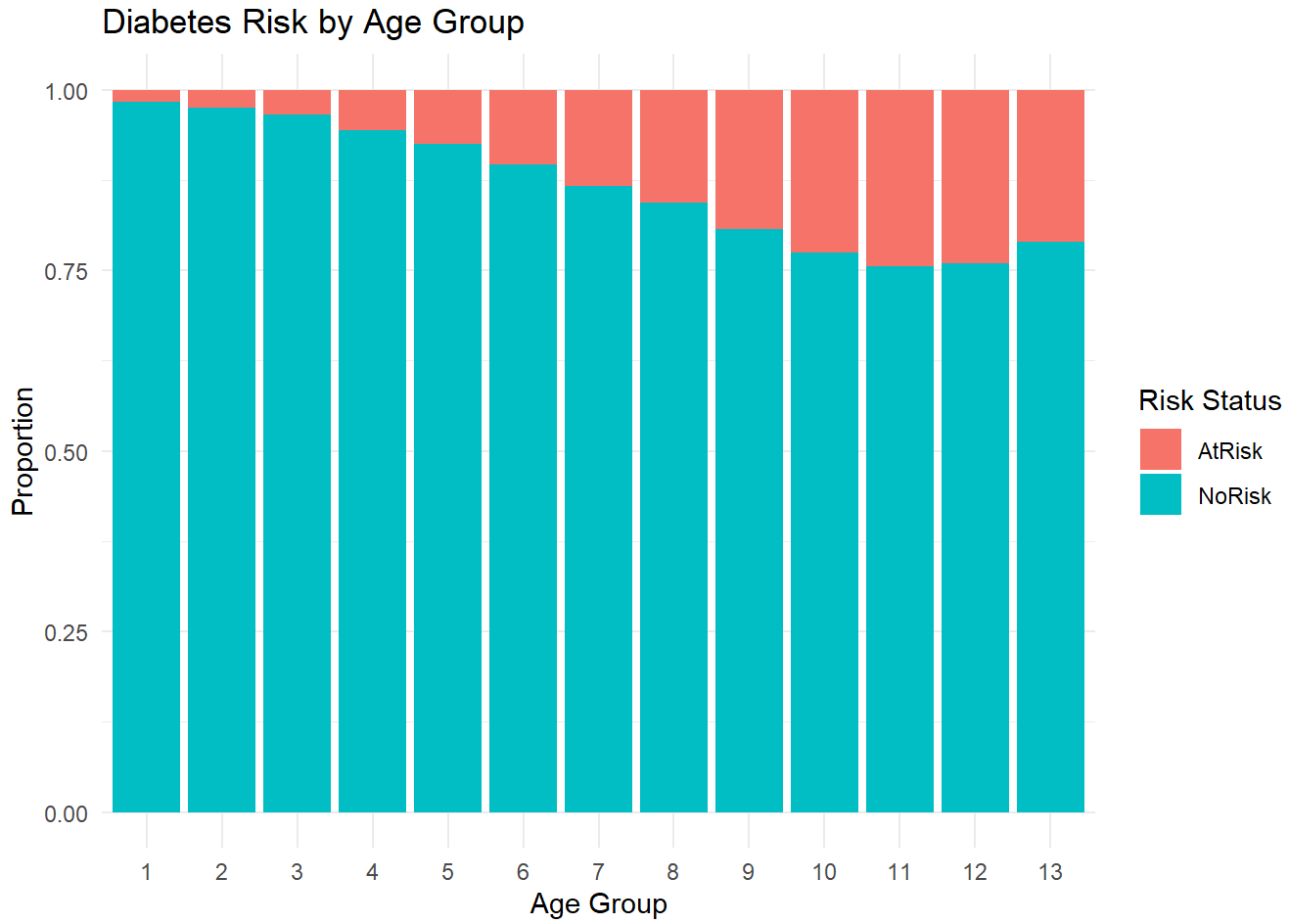
```
ggplot(demo_data, aes(x = Diabetes_Risk, fill = Diabetes_Risk)) +
  geom_bar() +
  labs(
```

```
    title = "Distribution of Diabetes Risk Status",
    x = "Diabetes Risk Status",
    y = "Number of Respondents",
    fill = "Risk Status"
  ) +
  theme_minimal()
```



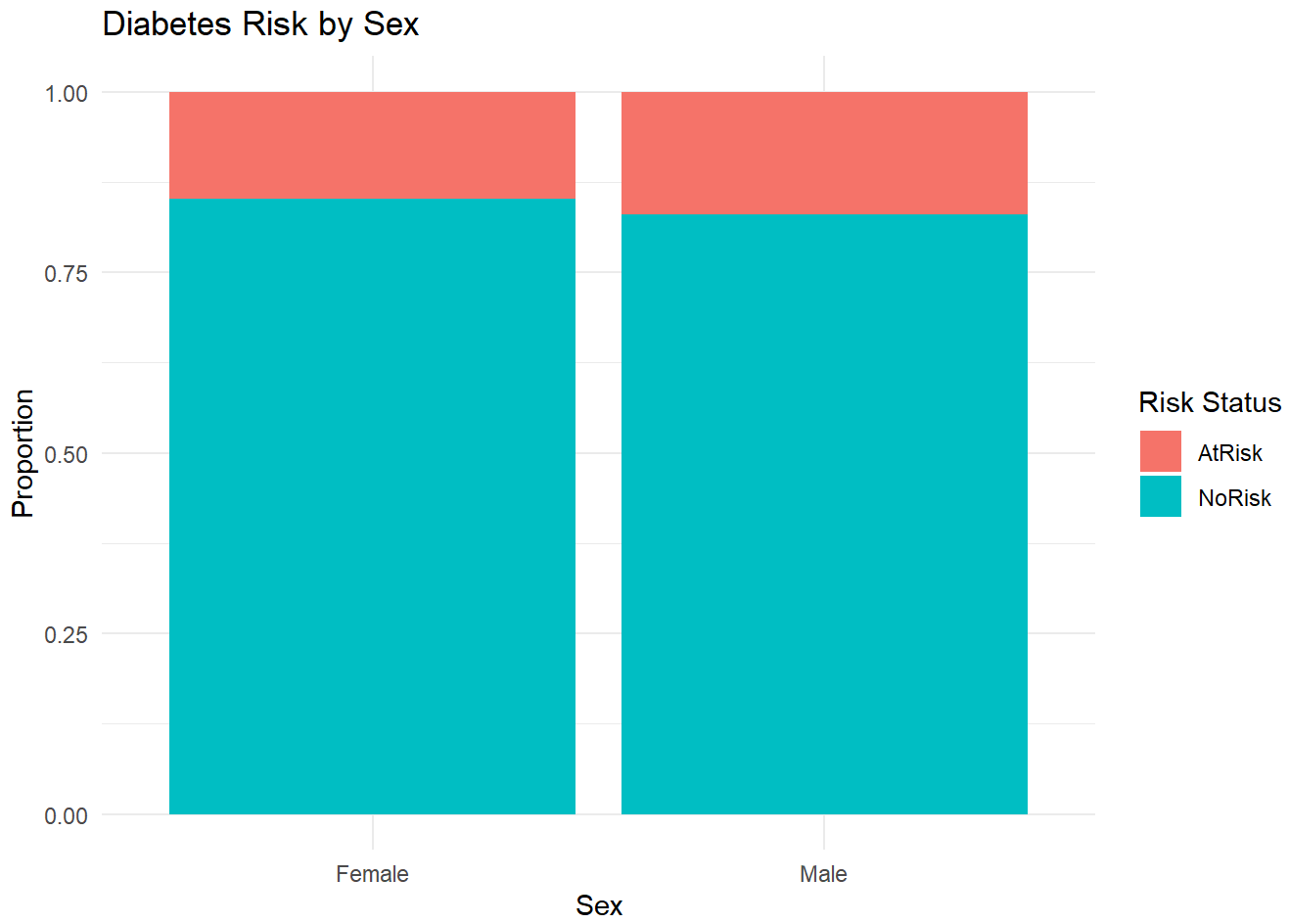
Diabetes Risk by Age

```r
ggplot(demo_data, aes(x = Age, fill = Diabetes_Risk)) +
  geom_bar(position = "fill") +
  labs(
    title = "Diabetes Risk by Age Group",
    x = "Age Group",
    y = "Proportion",
    fill = "Risk Status"
  ) +
  theme_minimal()
```



Diabetes Risk by Sex

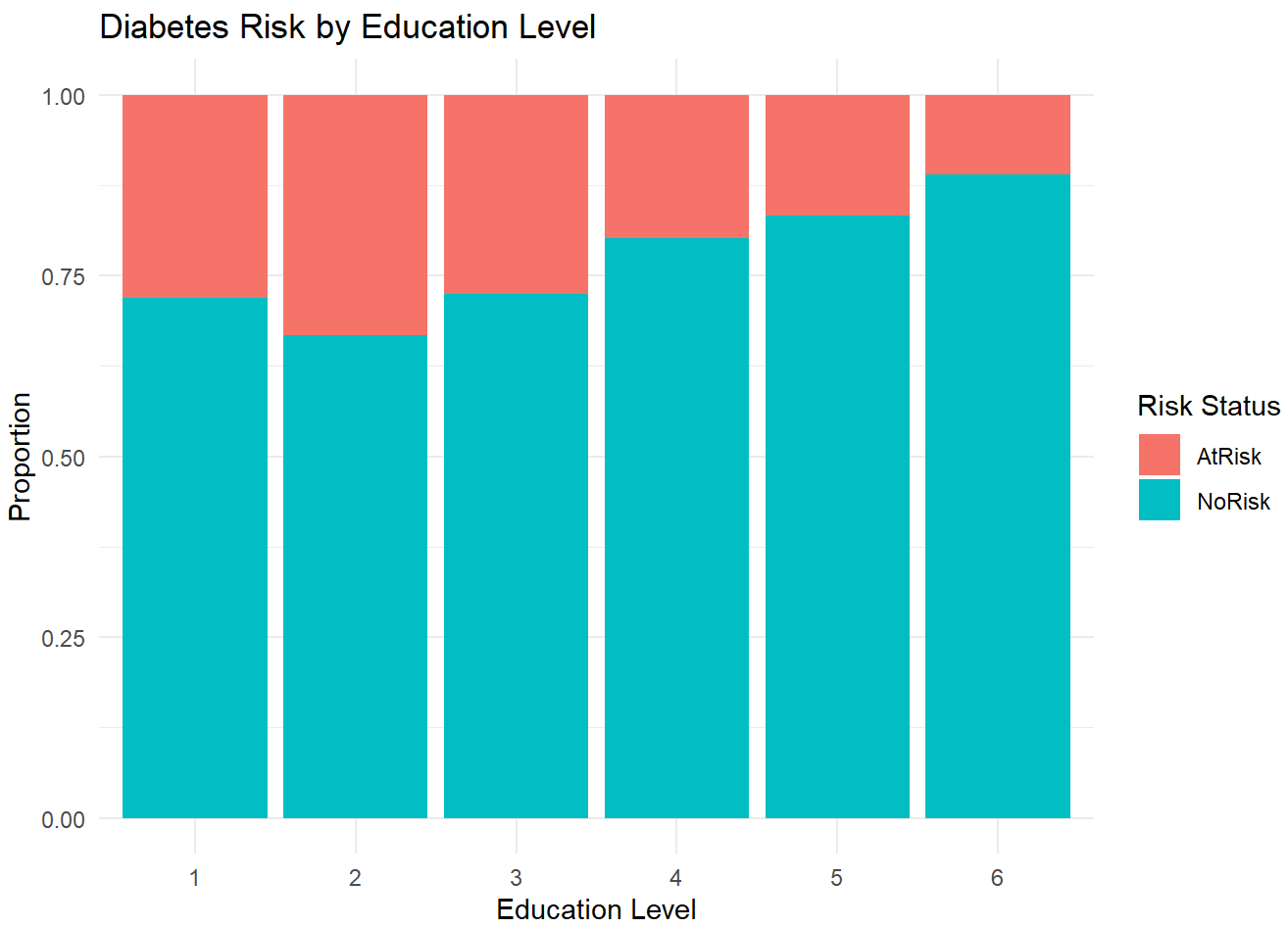
```
ggplot(demo_data, aes(x = Sex, fill = Diabetes_Risk)) +
  geom_bar(position = "fill") +
  labs(
    title = "Diabetes Risk by Sex",
    x = "Sex",
    y = "Proportion",
    fill = "Risk Status"
  ) +
  theme_minimal()
```



Diabetes Risk by Education

```
ggplot(demo_data, aes(x = Education, fill = Diabetes_Risk)) +
  geom_bar(position = "fill") +
  labs(
    title = "Diabetes Risk by Education Level",
    x = "Education Level",
    y = "Proportion",
    fill = "Risk Status"
```

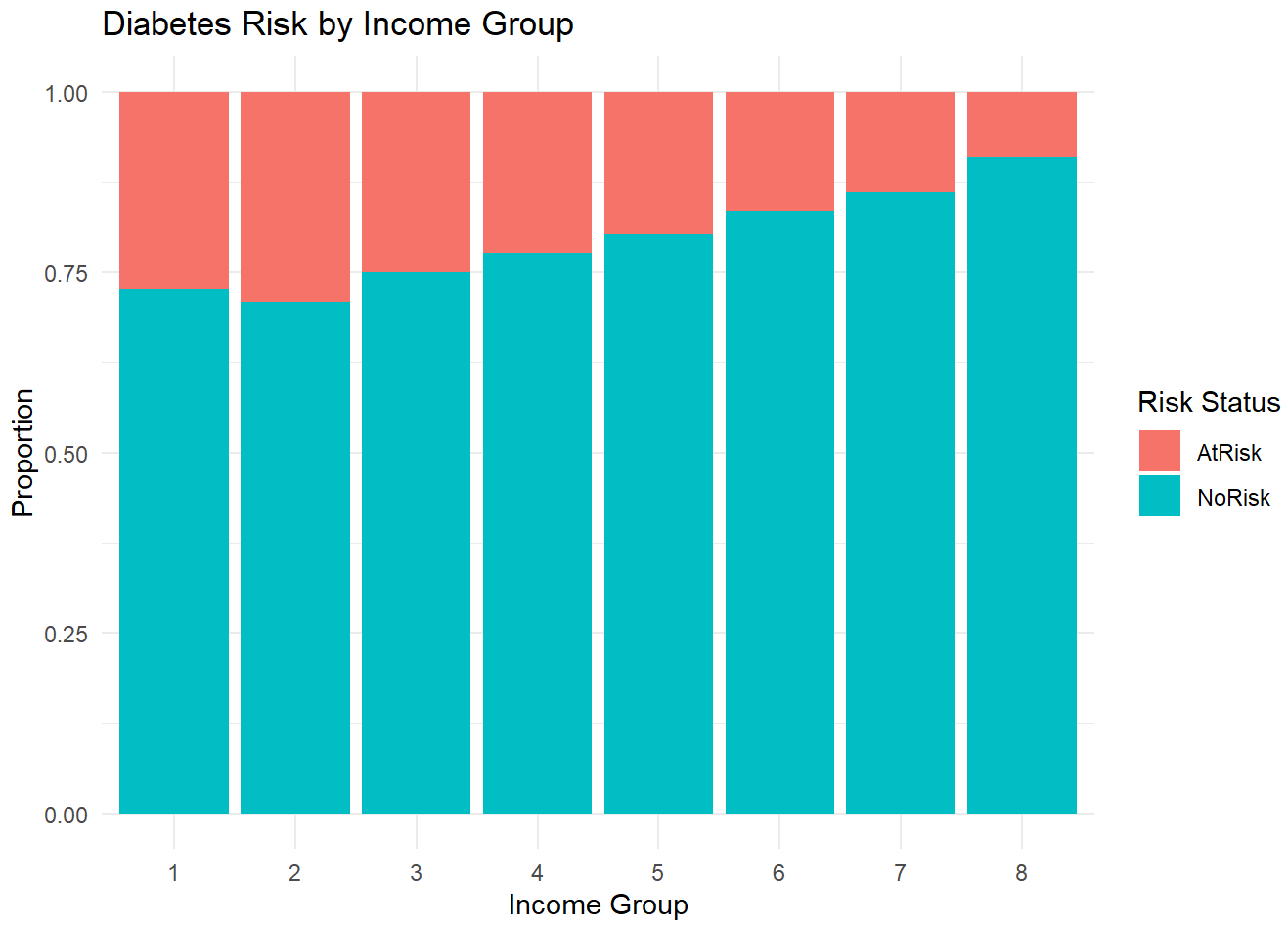
```
  ) +
  theme_minimal()
```



Diabetes Risk by Income

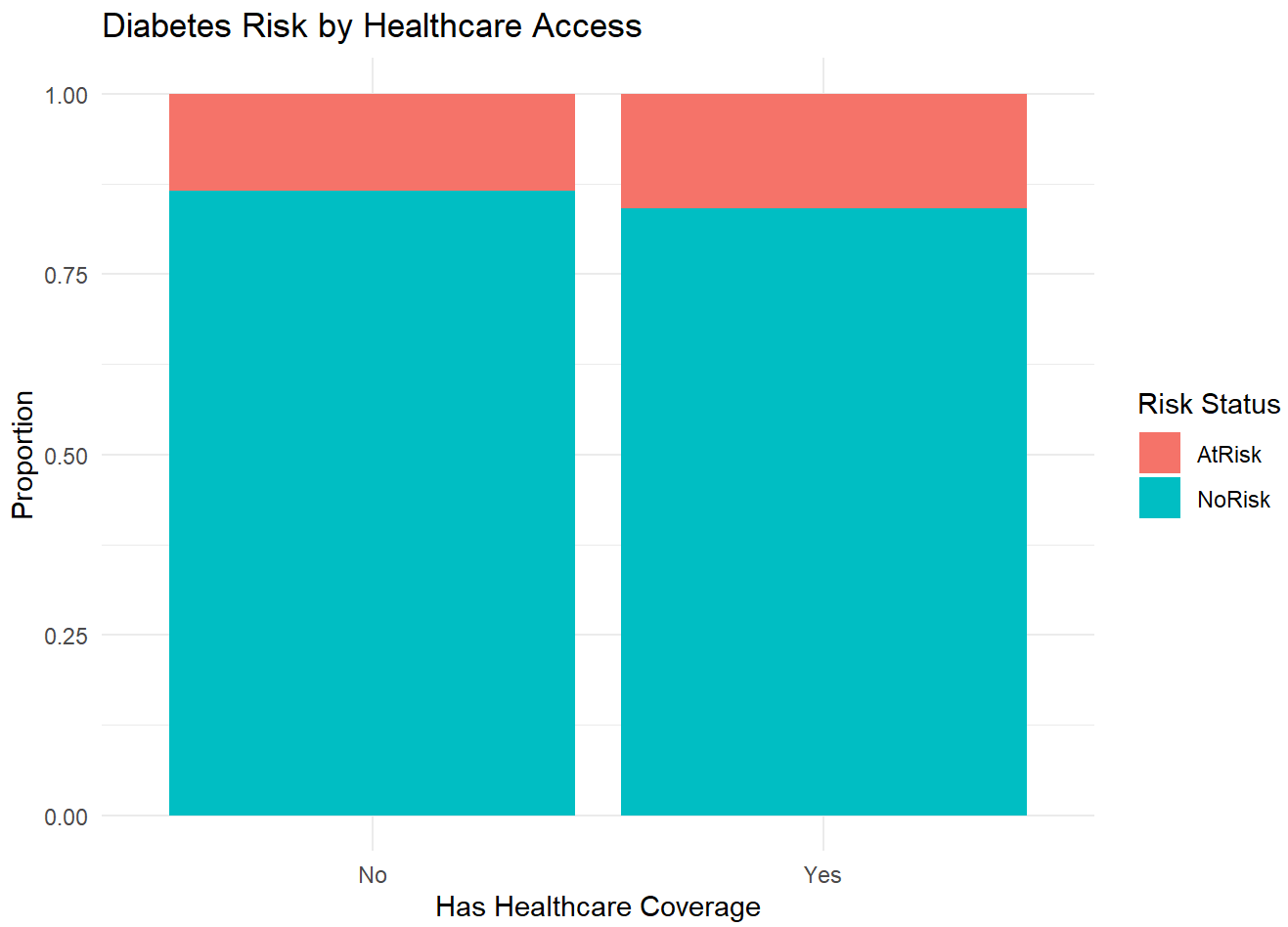
```
ggplot(demo_data, aes(x = Income, fill = Diabetes_Risk)) +
  geom_bar(position = "fill") +
  labs(
    title = "Diabetes Risk by Income Group",
```

```
    x = "Income Group",
    y = "Proportion",
    fill = "Risk Status"
  ) +
  theme_minimal()
```



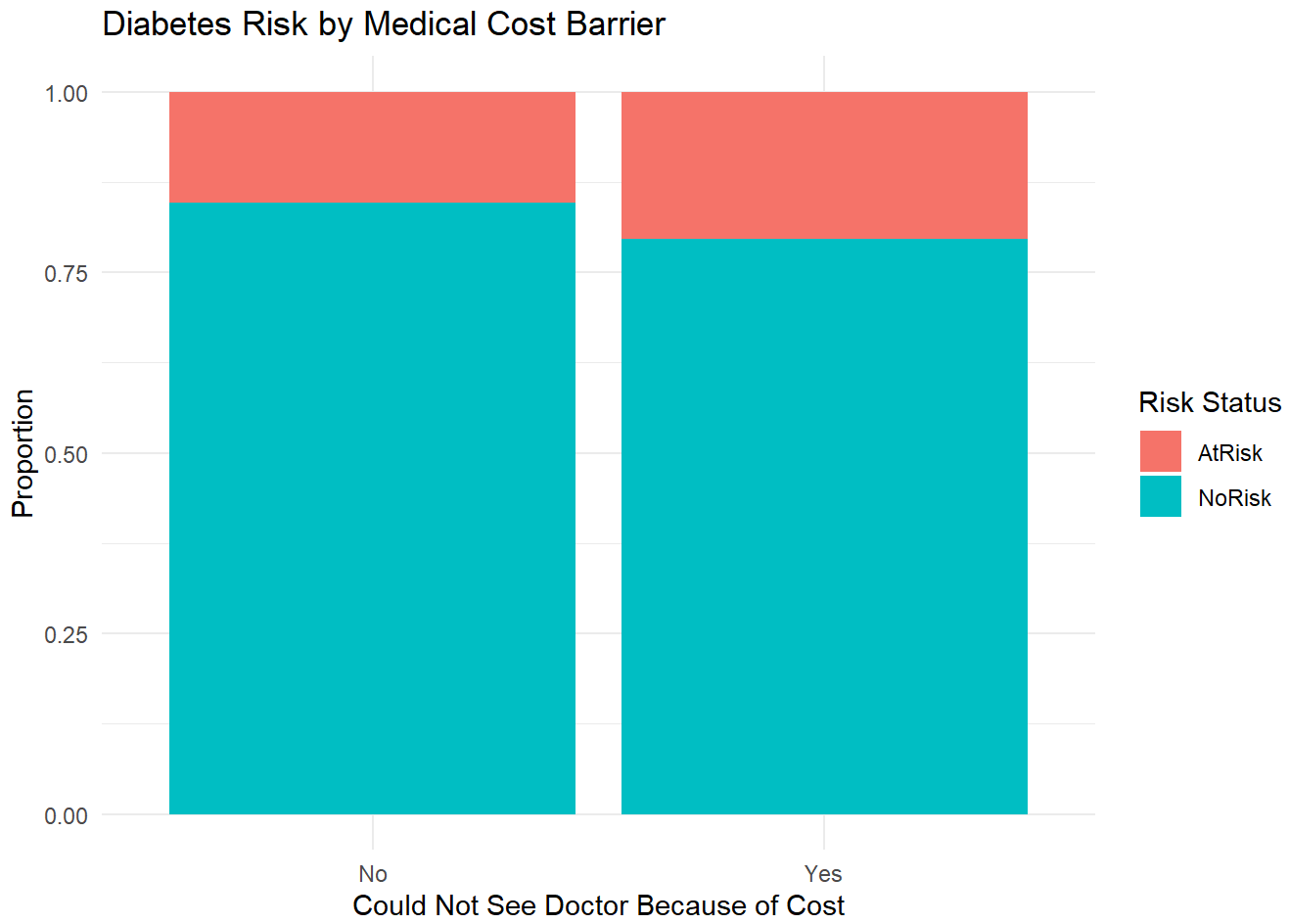
Diabetes Risk by Healthcare Access

```r
ggplot(demo_data, aes(x = AnyHealthcare, fill = Diabetes_Risk)) +
  geom_bar(position = "fill") +
  labs(
    title = "Diabetes Risk by Healthcare Access",
    x = "Has Healthcare Coverage",
    y = "Proportion",
    fill = "Risk Status"
  ) +
  theme_minimal()
```



Diabetes Risk by Medical Cost Barrier

```
ggplot(demo_data, aes(x = NoDocbcCost, fill = Diabetes_Risk)) +
  geom_bar(position = "fill") +
  labs(
    title = "Diabetes Risk by Medical Cost Barrier",
    x = "Could Not See Doctor Because of Cost",
    y = "Proportion",
    fill = "Risk Status"
  ) +
  theme_minimal()
```



Train and Test Split

The dataset has more than 250,000 observations, so a train and test split is appropriate. The split is stratified to preserve the class distribution.

```
set.seed(503)

train_index <- createDataPartition(
  demo_data$Diabetes_Risk,
```

```
  p = 0.80,
  list = FALSE
)

train_data <- demo_data[train_index, ]
test_data <- demo_data[-train_index, ]

prop.table(table(train_data$Diabetes_Risk)) * 100
```

```
  AtRisk   NoRisk 
15.75895 84.24105 
```

```
prop.table(table(test_data$Diabetes_Risk)) * 100
```

```
  AtRisk   NoRisk 
15.75835 84.24165 
```

Balanced Training Sample

The training data is imbalanced, so a balanced training sample is used to help the models learn both classes. The final evaluation still uses the original test set.

```
set.seed(503)

at_risk_train <- train_data %>%
  filter(Diabetes_Risk == "AtRisk")

no_risk_train <- train_data %>%
  filter(Diabetes_Risk == "NoRisk") %>%
  slice_sample(n = nrow(at_risk_train))

train_balanced <- bind_rows(at_risk_train, no_risk_train) %>%
  slice_sample(prop = 1)
```

```
table(train_balanced$Diabetes_Risk)
```

```
AtRisk NoRisk
 31982  31982
```

Cross Validation Setup

```
set.seed(503)

ctrl <- trainControl(
  method = "repeatedcv",
  number = 5,
  repeats = 2,
  classProbs = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final"
)
```

The models are tuned using sensitivity because this project is framed as a health screening problem. For screening, catching true at-risk cases is important.

Model 1: Logistic Regression

```
set.seed(503)

model_logit <- train(
  Diabetes_Risk ~ .,
  data = train_balanced,
  method = "glm",
  family = "binomial",
  trControl = ctrl,
  metric = "Sens"
```

```
)

model_logit
```

```
Generalized Linear Model

63964 samples
    6 predictor
    2 classes: 'AtRisk', 'NoRisk'

No pre-processing
Resampling: Cross-Validated (5 fold, repeated 2 times)
Summary of sample sizes: 51171, 51171, 51171, 51172, 51171, 51172, ...
Resampling results:

  ROC        Sens       Spec
  0.7090346  0.6967823  0.6061688
```

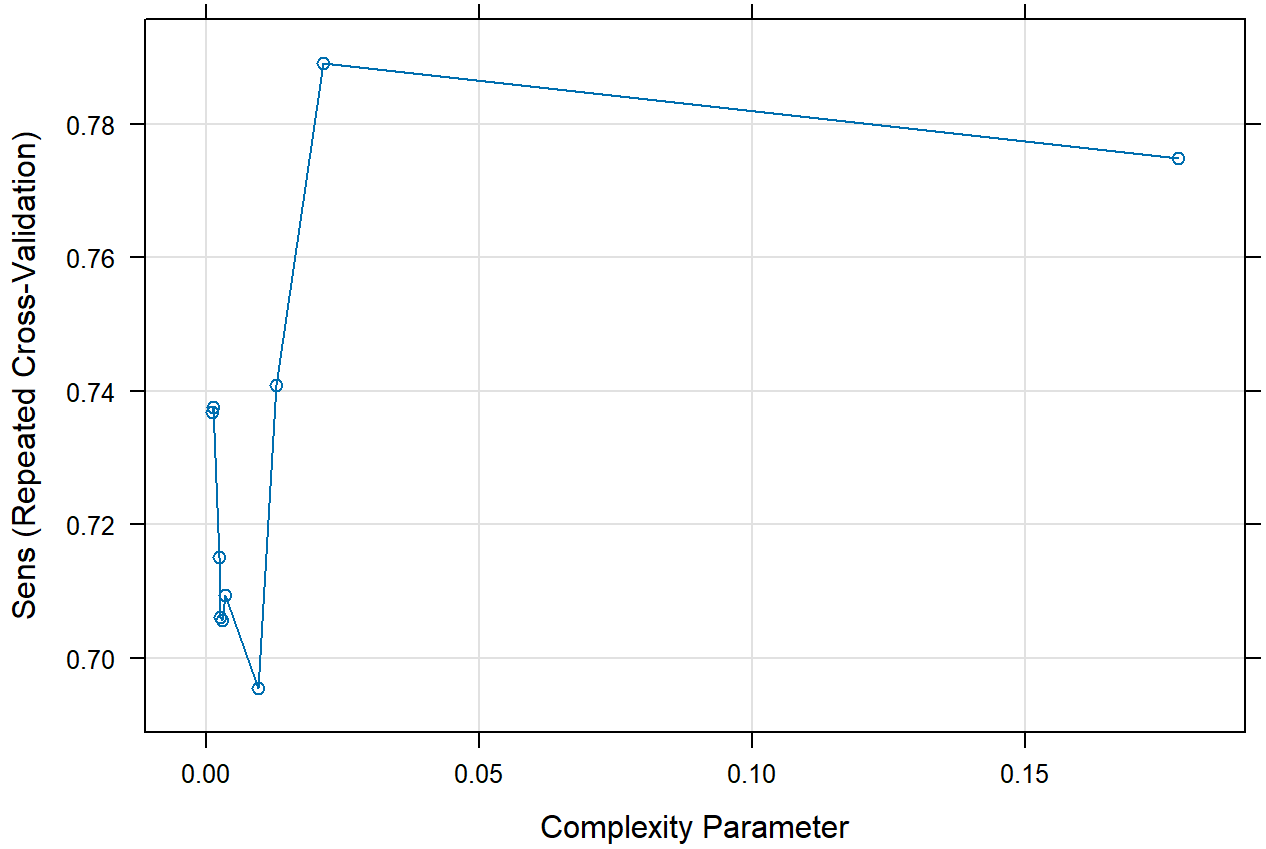
Logistic regression is used as a baseline model because it is simple and interpretable.

Model 2: Decision Tree

```
set.seed(503)

model_tree <- train(
  Diabetes_Risk ~ .,
  data = train_balanced,
  method = "rpart",
  trControl = ctrl,
  metric = "Sens",
  tuneLength = 10
)

model_tree
```

```
CART

63964 samples
    6 predictor
    2 classes: 'AtRisk', 'NoRisk'

No pre-processing
Resampling: Cross-Validated (5 fold, repeated 2 times)
Summary of sample sizes: 51171, 51171, 51171, 51172, 51171, 51172, ...
Resampling results across tuning parameters:

  cp           ROC        Sens       Spec
  0.001125633  0.6589195  0.7368044  0.5381776
  0.001141267  0.6589169  0.7376487  0.5375522
  0.002313802  0.6496439  0.7150733  0.5495271
  0.002470139  0.6478046  0.7061783  0.5557655
  0.002798449  0.6469818  0.7056622  0.5544838
  0.003345632  0.6401853  0.7093988  0.5444937
  0.009474079  0.6345158  0.6954668  0.5442899
  0.012772810  0.6241883  0.7409327  0.4884626
  0.021340129  0.6009037  0.7891155  0.4095271
  0.178037646  0.5513808  0.7747875  0.3279742

Sens was used to select the optimal model using the largest value.
The final value used for the model was cp = 0.02134013.
```

```
plot(model_tree)
```



The decision tree can capture nonlinear relationships and is easy to explain visually.

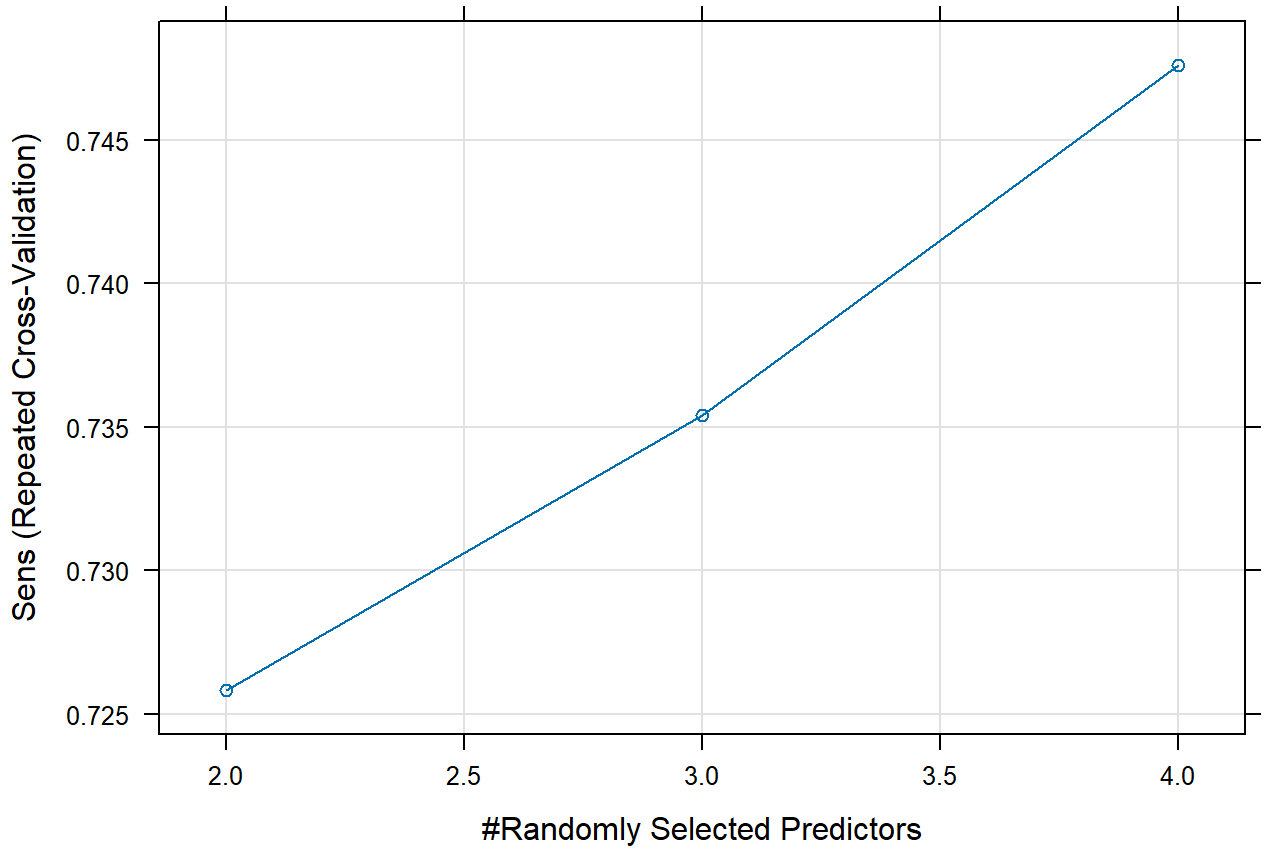
Model 3: Random Forest

```
set.seed(503)

model_rf <- train(
  Diabetes_Risk ~ .,
  data = train_balanced,
  method = "rf",
```

```
  trControl = ctrl,
  metric = "Sens",
  tuneGrid = data.frame(mtry = c(2, 3, 4)),
  ntree = 100
)

model_rf
```

```
Random Forest

63964 samples
    6 predictor
    2 classes: 'AtRisk', 'NoRisk'

No pre-processing
Resampling: Cross-Validated (5 fold, repeated 2 times)
Summary of sample sizes: 51171, 51171, 51171, 51172, 51171, 51172, ...
Resampling results across tuning parameters:

  mtry  ROC        Sens       Spec
  2     0.6844087  0.7258137  0.5533110
  3     0.6880481  0.7353984  0.5495124
  4     0.6899790  0.7476232  0.5402417

Sens was used to select the optimal model using the largest value.
The final value used for the model was mtry = 4.
```

```
plot(model_rf)
```



Random Forest is included because it can capture more complex relationships and provides variable importance results.

Cross Validation Comparison

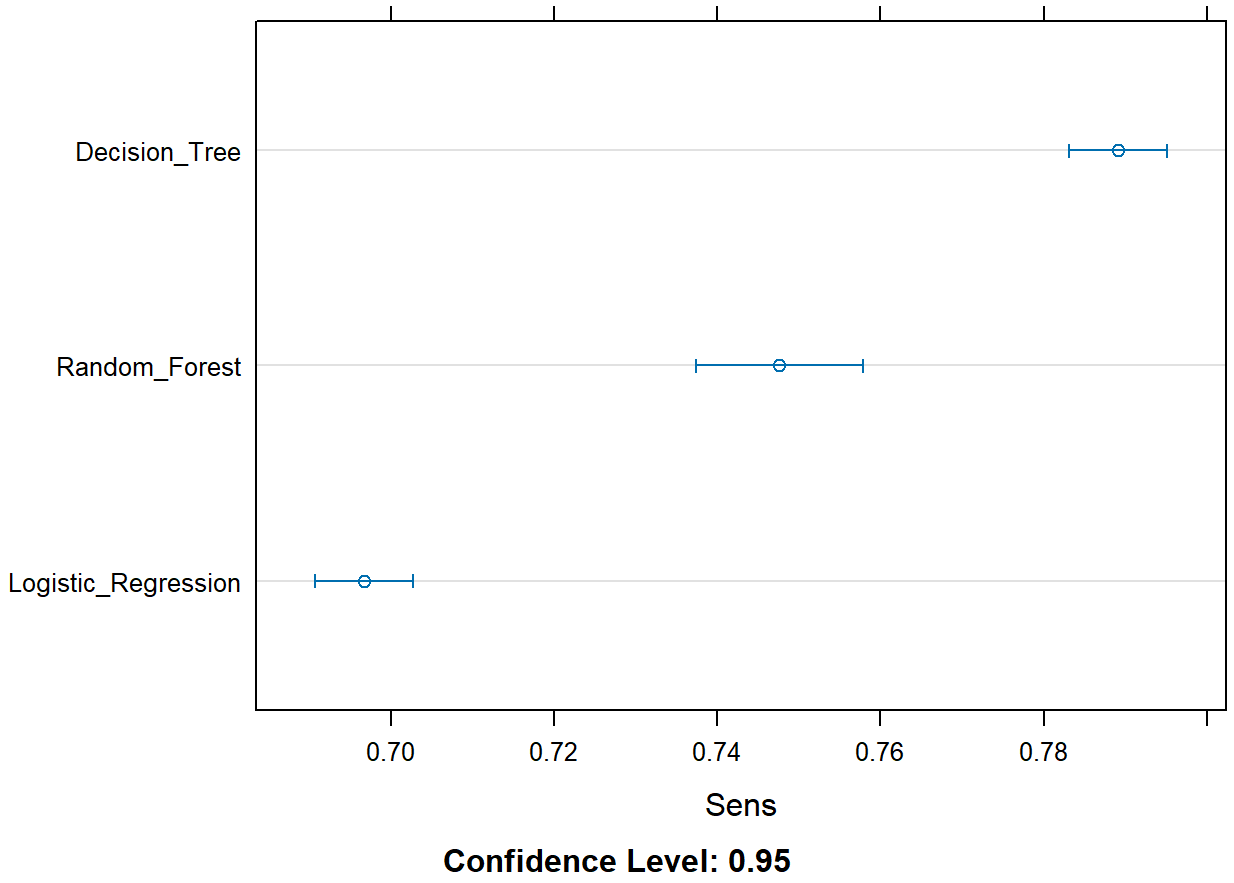
```
model_results <- resamples(
  list(
    Logistic_Regression = model_logit,
    Decision_Tree = model_tree,
    Random_Forest = model_rf
```

```
  )
)

summary(model_results)
```

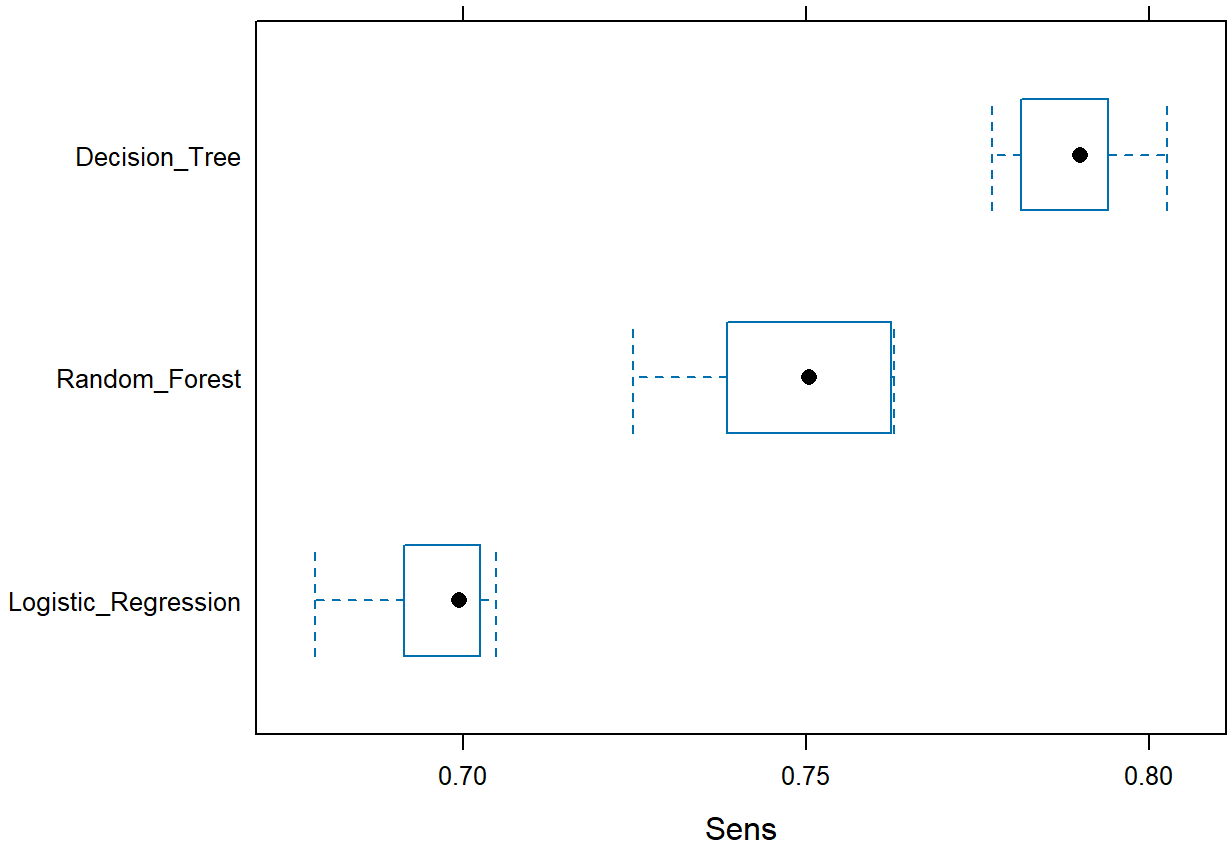
```
Call:
summary.resamples(object = model_results)

Models: Logistic_Regression, Decision_Tree, Random_Forest
Number of resamples: 10

ROC
                         Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
Logistic_Regression 0.7035058 0.7055117 0.7087982 0.7090346 0.7122109 0.7157723
Decision_Tree       0.5858828 0.5888455 0.6007064 0.6009037 0.6105664 0.6177120
Random_Forest       0.6846959 0.6872234 0.6890719 0.6899790 0.6932272 0.6959411
                    NA's
Logistic_Regression    0
Decision_Tree          0
Random_Forest          0

Sens
                         Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
Logistic_Regression 0.6785491 0.6932038 0.6994997 0.6967823 0.7025052 0.7048617
Decision_Tree       0.7770482 0.7822333 0.7898687 0.7891155 0.7939809 0.8025637
Random_Forest       0.7248280 0.7386564 0.7504690 0.7476232 0.7610320 0.7628576
                    NA's
Logistic_Regression    0
Decision_Tree          0
Random_Forest          0

Spec
                         Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
Logistic_Regression 0.5978737 0.6011179 0.6059090 0.6061688 0.6096846 0.6187275
Decision_Tree       0.3752345 0.3857338 0.4122889 0.4095271 0.4343219 0.4372362
Random_Forest       0.5225141 0.5318118 0.5408068 0.5402417 0.5475474 0.5581614
                    NA's
```

```
Logistic_Regression    0
Decision_Tree          0
Random_Forest          0
```

```
dotplot(model_results, metric = "Sens")
```




```
bwplot(model_results, metric = "Sens")
```



Test Set Evaluation

```
calculate_metrics <- function(predicted, actual, model_name) {
  cm <- confusionMatrix(predicted, actual, positive = "AtRisk")

  accuracy <- as.numeric(cm$overall["Accuracy"])
  kappa <- as.numeric(cm$overall["Kappa"])
  sensitivity <- as.numeric(cm$byClass["Sensitivity"])
  specificity <- as.numeric(cm$byClass["Specificity"])
```

```
  precision <- as.numeric(cm$byClass["Pos Pred Value"])
  f1 <- as.numeric(cm$byClass["F1"])
  balanced_accuracy <- mean(c(sensitivity, specificity), na.rm = TRUE)

  data.frame(
    Model = model_name,
    Accuracy = round(accuracy, 4),
    Kappa = round(kappa, 4),
    Sensitivity_Recall = round(sensitivity, 4),
    Specificity = round(specificity, 4),
    Precision = round(precision, 4),
    F1 = round(f1, 4),
    Balanced_Accuracy = round(balanced_accuracy, 4)
  )
}
```

```
pred_logit <- predict(model_logit, newdata = test_data)
pred_tree <- predict(model_tree, newdata = test_data)
pred_rf <- predict(model_rf, newdata = test_data)

cm_logit <- confusionMatrix(pred_logit, test_data$Diabetes_Risk, positive = "AtRisk")
cm_tree <- confusionMatrix(pred_tree, test_data$Diabetes_Risk, positive = "AtRisk")
cm_rf <- confusionMatrix(pred_rf, test_data$Diabetes_Risk, positive = "AtRisk")

cm_logit
```

```
Confusion Matrix and Statistics

          Reference
Prediction AtRisk NoRisk
    AtRisk   5529  16635
    NoRisk   2466  26105

               Accuracy : 0.6235
                 95% CI : (0.6193, 0.6277)
    No Information Rate : 0.8424
    P-Value [Acc > NIR] : 1
```

```
                 Kappa : 0.1757

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.6916
            Specificity : 0.6108
         Pos Pred Value : 0.2495
         Neg Pred Value : 0.9137
             Prevalence : 0.1576
         Detection Rate : 0.1090
   Detection Prevalence : 0.4369
      Balanced Accuracy : 0.6512

       'Positive' Class : AtRisk
```

```
cm_tree
```

```
Confusion Matrix and Statistics

          Reference
Prediction AtRisk NoRisk
    AtRisk   6341  26158
    NoRisk   1654  16582

               Accuracy : 0.4518
                 95% CI : (0.4475, 0.4562)
    No Information Rate : 0.8424
    P-Value [Acc > NIR] : 1

                  Kappa : 0.0806

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.7931
            Specificity : 0.3880
```

```
         Pos Pred Value : 0.1951
         Neg Pred Value : 0.9093
             Prevalence : 0.1576
         Detection Rate : 0.1250
   Detection Prevalence : 0.6406
      Balanced Accuracy : 0.5905

       'Positive' Class : AtRisk
```

```
cm_rf
```

```
Confusion Matrix and Statistics

          Reference
Prediction AtRisk NoRisk
    AtRisk   5940  19437
    NoRisk   2055  23303

               Accuracy : 0.5764
                 95% CI : (0.5721, 0.5807)
    No Information Rate : 0.8424
    P-Value [Acc > NIR] : 1

                  Kappa : 0.153

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.7430
            Specificity : 0.5452
         Pos Pred Value : 0.2341
         Neg Pred Value : 0.9190
             Prevalence : 0.1576
         Detection Rate : 0.1171
   Detection Prevalence : 0.5002
      Balanced Accuracy : 0.6441
```

```
'Positive' Class : AtRisk
```

Model Comparison Table

```
comparison_table <- bind_rows(
  calculate_metrics(pred_logit, test_data$Diabetes_Risk, "Logistic Regression"),
  calculate_metrics(pred_tree, test_data$Diabetes_Risk, "Decision Tree"),
  calculate_metrics(pred_rf, test_data$Diabetes_Risk, "Random Forest")
)

kable(comparison_table, caption = "Test Set Model Comparison")
```

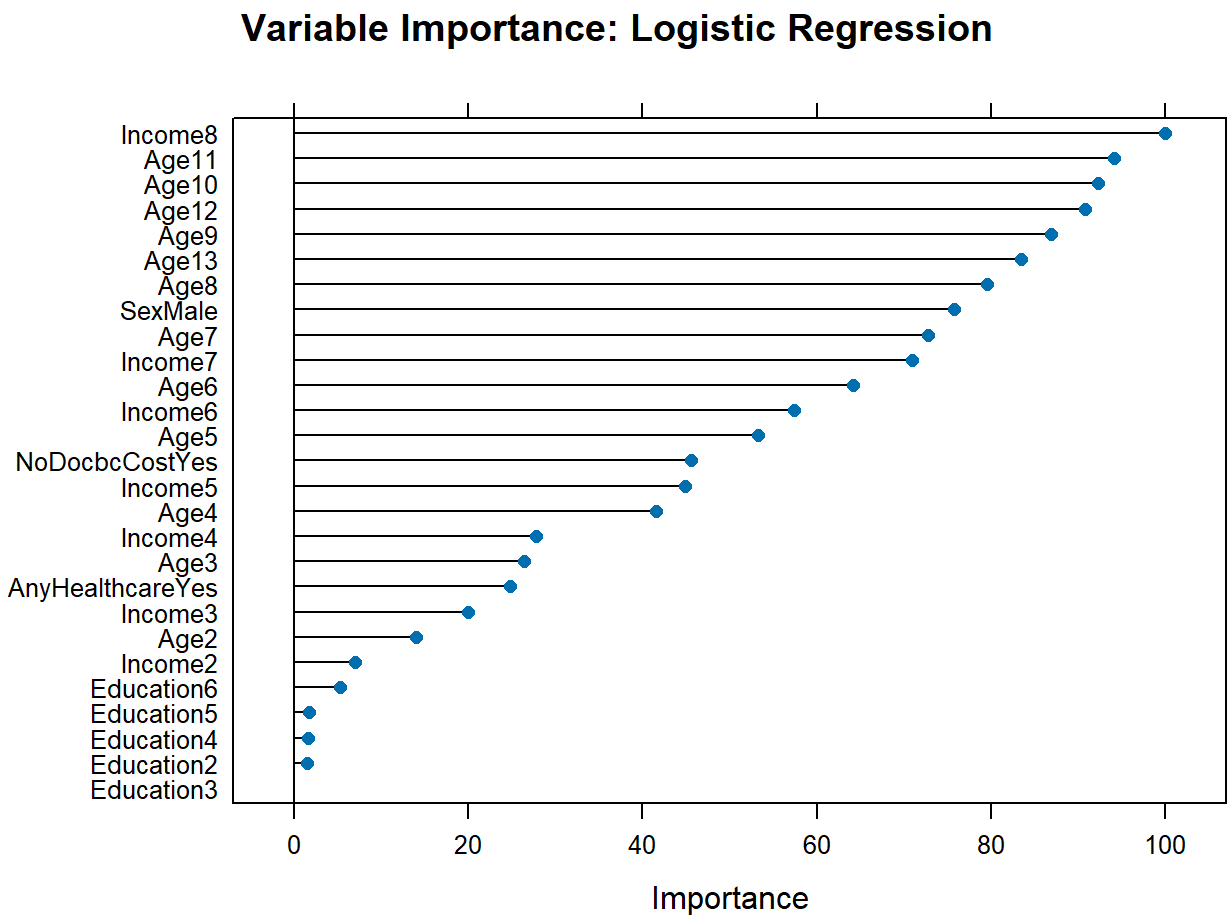
Test Set Model Comparison

Model	Accuracy	Kappa	Sensitivity_Recall	Specificity	Precision	F1	Balanced_Accuracy
Logistic Regression	0.6235	0.1757	0.6916	0.6108	0.2495	0.3667	0.6512
Decision Tree	0.4518	0.0806	0.7931	0.3880	0.1951	0.3132	0.5905
Random Forest	0.5764	0.1530	0.7430	0.5452	0.2341	0.3560	0.6441

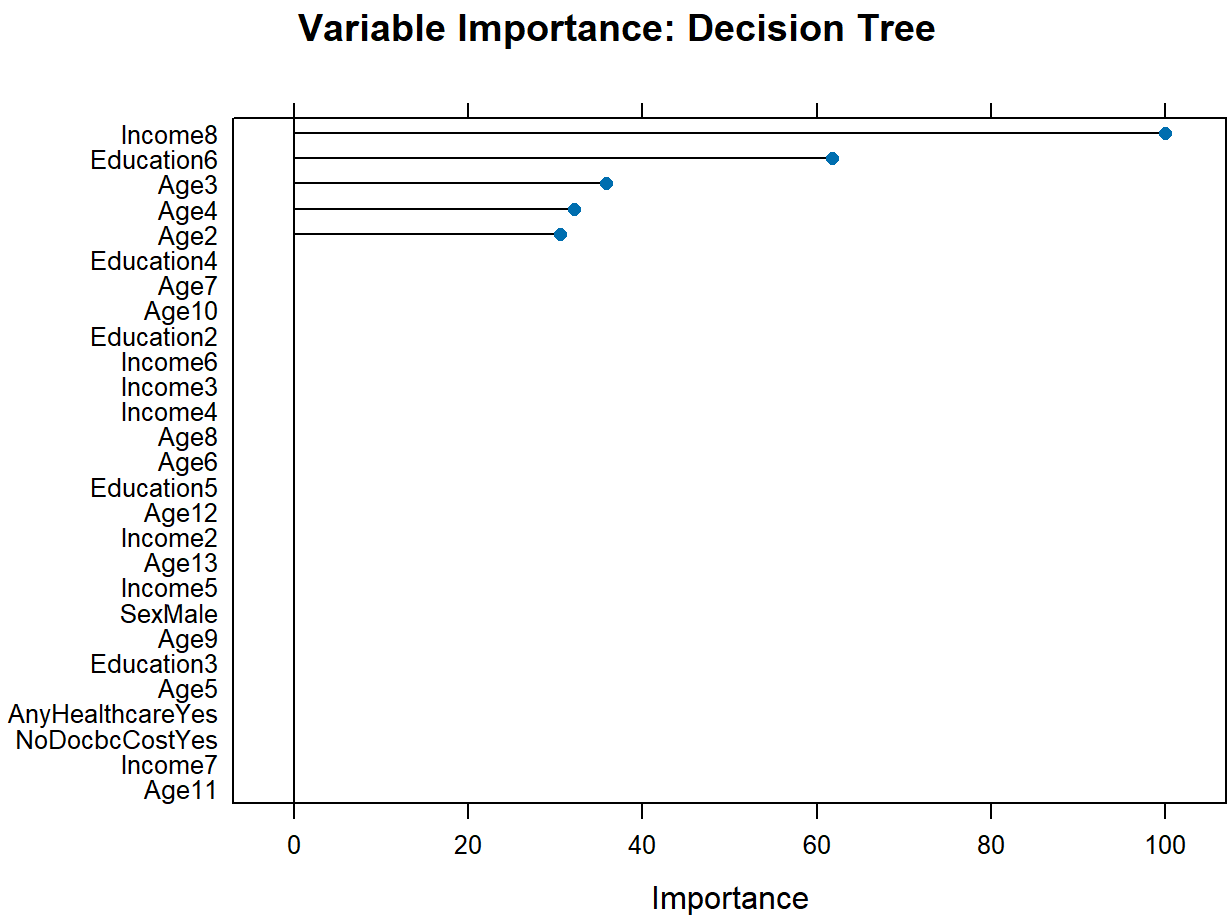
This table reports multiple metrics instead of only accuracy. This is important because the dataset is imbalanced.

Variable Importance

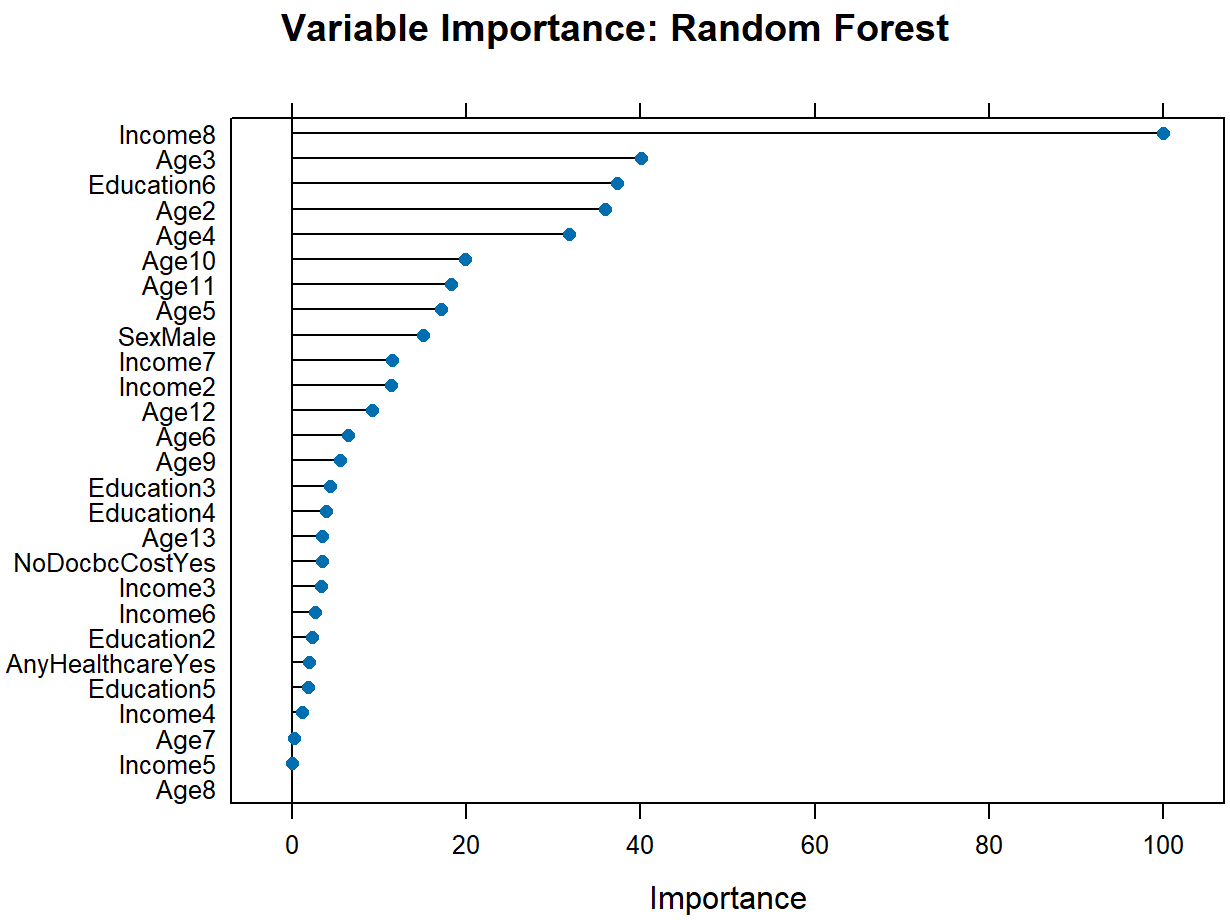
```
importance_logit <- varImp(model_logit)
importance_tree <- varImp(model_tree)
importance_rf <- varImp(model_rf)

plot(importance_logit, main = "Variable Importance: Logistic Regression")
```




```
plot(importance_tree, main = "Variable Importance: Decision Tree")
```




```
plot(importance_rf, main = "Variable Importance: Random Forest")
```



Variable importance helps explain which demographic and socioeconomic variables are most useful for identifying diabetes risk.

Discussion

This analysis used demographic and socioeconomic variables to predict diabetes risk. The target variable was converted from three classes into a binary screening target because the goal is to identify people who may be at risk and may need follow-up care.

The dataset is imbalanced, so accuracy alone is not enough to evaluate model quality. A model could appear accurate by predicting the majority class too often. For that reason, this analysis reports sensitivity, specificity, F1 score, balanced accuracy, and confusion matrices.

Sensitivity is important because this is a screening problem. If the model misses too many at-risk individuals, it would not be useful for early identification. Specificity is also important because too many false positives could create unnecessary follow-up work.

Conclusion

The demographic feature group provides useful information for early diabetes risk screening. Variables such as age, sex, education, income, healthcare access, and medical cost barriers are practical because they can be collected without laboratory testing.

The results from this demographic analysis can be combined with the lifestyle feature analysis to support the final group project.